# ShopSmart Grounded Product Assistant

Laboratory Report & Architecture Design Note — Week 5 Guided Challenge

| | | |
|---|---|---|
| **Student Name:** Andrik Brenth Canlas | **Class / Section:** BSIT 4-A | **Subject:** Advanced Software Construction |
| **Repository:** github.com/canlasandrikbrenth-source/Canlas-ShopSmart | **Provider:** Google Gemini API (Grounded Flash Tier) | **Status:** All 12 Checks Passing |

## 1. End-to-End Request Flow & Architecture

```
[ React UI: AssistantPage.jsx ] (1. Submits inquiry e.g. 'Which keyboard is below ■2,000?')
   ↓  fetch POST /api/assistant/questions { question }
[ Express Route: backend/src/routes/assistant.js ] (2. Zod validation 3-300 chars, Rate limit 20 req/min)
   ↓  authoritative active products query from PostgreSQL via Prisma Client (or memoryStore fallback)
[ Catalog Retrieval & Filter: assistant.js -> rankProducts() ]
   ■■■ No in-stock match ■■■ Returns honest no-match response immediately (Provider Bypassed)
   ■■■ Candidates qualified ■■ Formats safe public fields: id, name, category, price, stock, description
       ↓
[ AI Provider Integration: backend/src/services/aiProvider.js -> generateAssistantDraft() ]
   ■■■ Provider Key: Kept strictly in backend/.env; passed via x-goog-api-key header (not URL/logs)
   ■■■ Timeout Protection: AbortSignal.timeout(5000ms)
   ■■■ Failure / Timeout / 503 ■■■ Seamless fallback to buildRuleBasedResponse()
   ■■■ Valid JSON Draft ■■■ Contains natural explanation & recommendedProductIds
       ↓
[ Server-Side Grounding & Verification: assistant.js -> buildAssistantResponse() ]
   ■■■ Intercepts recommendedProductIds; strictly rejects any ID not in pre-retrieved candidate set
   ■■■ Constructs recommendedProducts & sources exclusively from authoritative catalog records
       ↓
[ Final Response Payload ] ■■■ HTTP 200 { answer, recommendedProducts, sources }
   ↓
[ React UI Presentation ] ■■■ Renders natural answer, working /products/:id links, and verified sources
```

## 2. Five Core Design Decisions

**1. Product fields sent to provider:** id, name, category, price, stock, and description. *Rationale:* These fields provide sufficient factual context to evaluate category match, budget fit, and stock availability. Internal metadata (timestamps, relational keys, margins) and customer PII are strictly excluded. Database Decimal values are explicitly converted to standard JavaScript numbers.

**2. Preventing invented product IDs or sources:** All LLM outputs are treated strictly as untrusted draft text. The server checks model-returned IDs against an in-memory Map of pre-retrieved candidates (candidateMap = new Map(candidates.map(c => [c.id, c]))). Any unknown ID is rejected. Crucially, the final recommendedProducts and sources arrays are built directly by the server from database records, guaranteeing that all IDs, titles, prices, and links (/products/:id) are genuine.

**3. Catalog with no match:** When catalog retrieval yields zero matching in-stock products, **the AI provider is not called**. Calling an LLM without qualifying catalog facts wastes latency, burns quota, and invites hallucination. The server instantly returns an honest, deterministic response: { answer: 'I could not find an in-stock product matching that request.', recommendedProducts: [], sources: [] }.

**4. Provider unavailability & fallback:** If Gemini times out (>5000ms), encounters rate limits (429), spikes to 503 high demand, or if GEMINI_API_KEY is omitted, the error is safely caught without leaking credentials. The server immediately invokes buildRuleBasedResponse(). The rule engine still parses budgets (parseBudget), filters in-stock items, ranks keyword matches, and produces an accurate recommendation with formatted prices (■1,850) and working links.

**5. Public catalog data & permission boundaries:** Product catalog data (names, descriptions, active prices, stock levels) is public read-only information accessible without authentication. Privileged operations (editing products, updating stock, managing orders) require authenticated JWT/session tokens and RBAC middleware. The assistant endpoint (/api/assistant/questions) only reads public catalog facts and never receives or forwards private customer records to any AI provider.

## 3. Demonstrated Test Cases (6 Required Scenarios)

| Case | Input / Action | Expected Observation | Observed Result | Status |
|---|---|---|---|---|
| **1. Catalog Match** | *'Which keyboard is below ■2,000?'* | Real in-stock keyboard linked; price supported (■1,850). | HTTP 200. Recommends Compact Mechanical Keyboard (ID 2, ■1,850) with valid link and source #2. | **PASS** |
| **2. No Match** | *'Which sneakers are below ■2,000?'* | Honest no-match; no invented product or source. Provider bypassed. | HTTP 200. 'I could not find an in-stock product matching that request.' Zero products/sources. | **PASS** |
| **3. Invalid Input** | Question shorter than 3 chars (e.g. *'hi'*) | HTTP 422 with validation error. | HTTP 422. INVALID_QUESTION: 'Question must contain between 3 and 300 characters.' | **PASS** |
| **4. Availability** | Test fixture with keyboard stock: 0 | Not recommended as available; excluded from candidates. | HTTP 200. Out-of-stock keyboard excluded during retrieval; returns honest no-match. | **PASS** |
| **5. Provider Failure** | Invalid key or simulated provider timeout (>5000ms) | No endless loading; clear rule-based fallback response. | HTTP 200. Fallback triggers in <1ms: 'I recommend the Compact Mechanical Keyboard for ■1,850.' | **PASS** |
| **6. Source Check** | Inspect all returned product links and source IDs | Each points to an authoritative catalog record. | Verified: sources[0].id === 2 ('Compact Mechanical Keyboard'). Links point to /products/2. | **PASS** |

## 4. Automated Verification Log (npm run check)

```
> shopsmart-starter@1.0.0 check
> npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build
> shopsmart-frontend@1.0.0 lint  ✓ passed    > shopsmart-backend@1.0.0 lint   ✓ passed
> shopsmart-frontend@1.0.0 test  ✓ src/utils/cart.test.js (7 tests passed)
> shopsmart-backend@1.0.0 test (node --test)
✔ parseBudget extracts a peso budget (0.84ms)            ✔ rankProducts respects the budget (0.75ms)
✔ rankProducts excludes out-of-stock items (0.14ms)      ✔ rankProducts & assistant return honest no-match (0.37ms)
✔ questions < 3 chars fail validation HTTP 422 (1.27ms)   ✔ assistant response contains traceable sources (13.75ms)
✔ assistant gracefully falls back on provider fail (0.20ms)✔ buildAssistantResponse rejects hallucinated IDs (0.20ms)
✔ buildRuleBasedResponse builds grounded price (0.18ms)   ✔ a valid classroom order passes validation (2.42ms)
✔ duplicate products fail order validation (0.28ms)       ✔ an oversized assistant question fails validation (0.33ms)
■ tests 12 | pass 12 | fail 0 | cancelled 0
> shopsmart-frontend@1.0.0 build (vite build) -> dist/index.html (0.50 kB), dist/assets/index.js (274.31 kB) ✓ built in 824ms
```

## 5. Reflection & Technical Issue Resolved

**Individual Contribution:** Architected the server-side catalog retrieval pipeline, isolated provider credentials using secure x-goog-api-key request headers, engineered the hallucination verification layer that reconciles model recommendations against authoritative catalog data, and updated the React frontend to display visible authoritative sources.

**Technical Issue Resolved:** During initial testing with live Gemini API models, transient upstream demand spikes returned intermittent HTTP 503 errors. In an ungrounded architecture, this would crash the server route or freeze the UI loading spinner. I resolved this by implementing an AbortSignal.timeout(5000ms) guard and an automatic fallback to buildRuleBasedResponse(). The assistant now remains 100% operational, immediately returning factual catalog recommendations and working links even under total AI provider downtime.